

Computer Networks (2025-2026)

Part 3: Transport Layer

Jeroen Famaey

Andrei Belogaev

{jeroen.famaey, andrei.belogaev}@uantwerpen.be

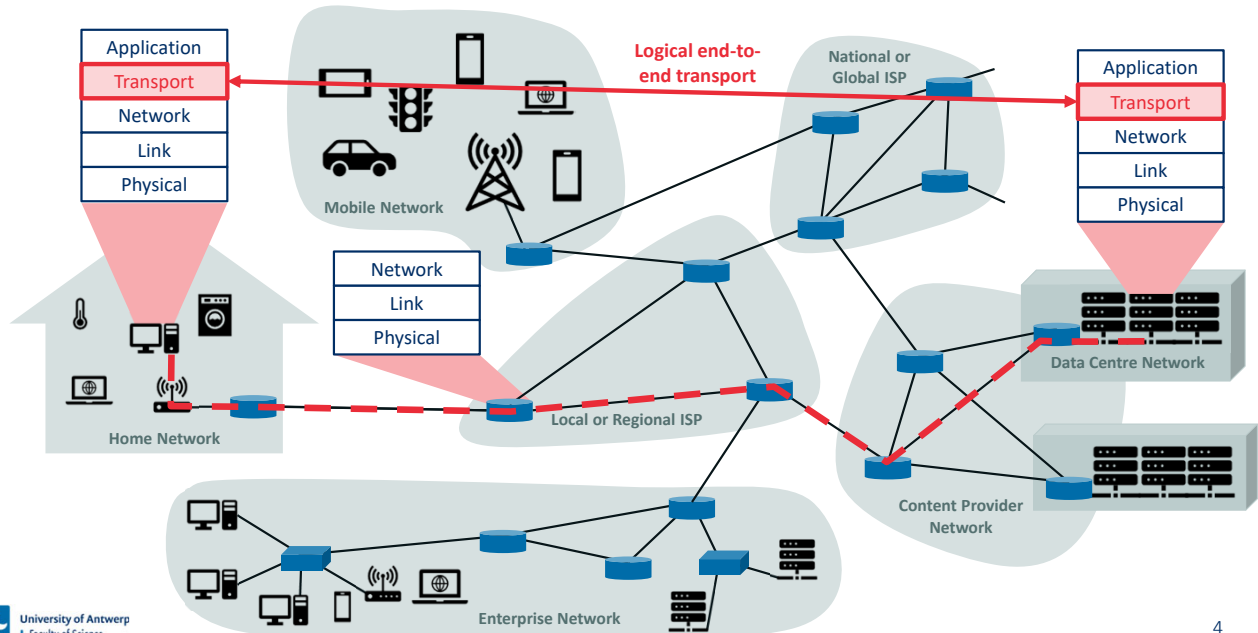
Table of contents

1. Transport-layer services
2. User Datagram Protocol (UDP)
3. Reliable data transfer
4. Transport Control Protocol (TCP)
5. TCP congestion control

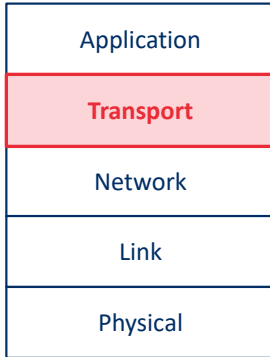
Transport-layer services



Transport layer provides a logical connection between applications

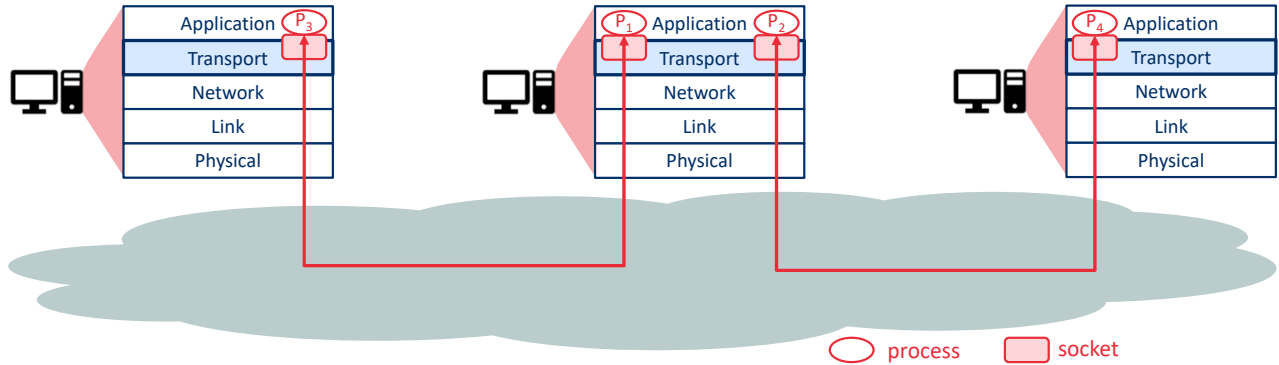


The Internet's transport Layer services



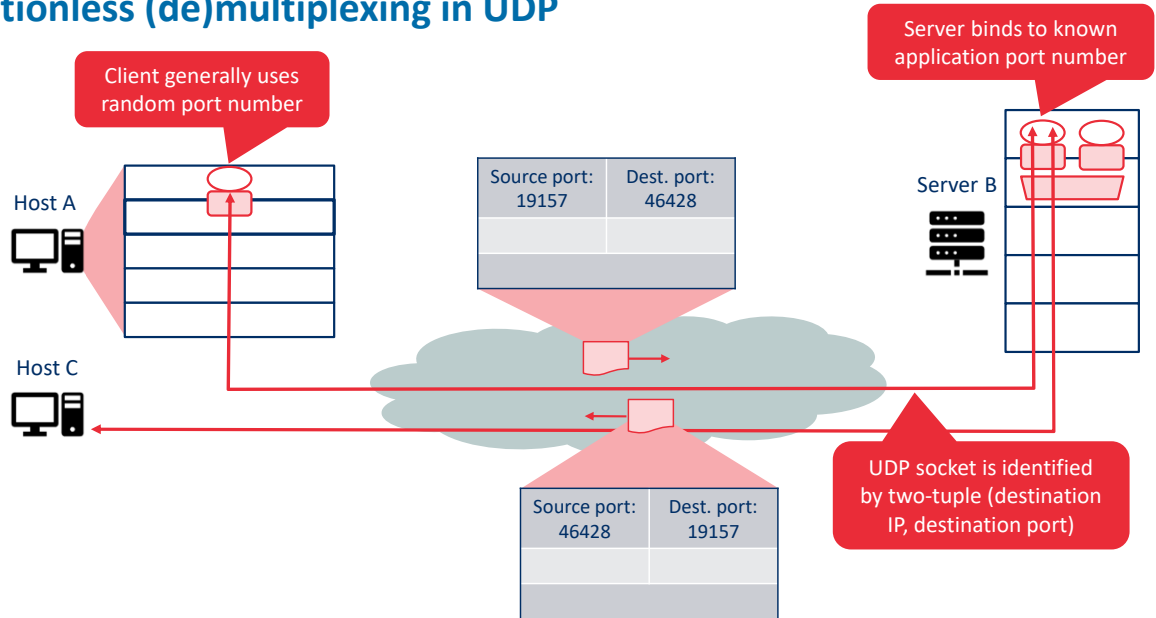
- Transports application-layer message between end-systems
- Transport-layer packets are referred to as **segments**
- **General services**
 - (De)multiplexing: Deliver segments to the correct process running on the end host
 - Error checking: Check the integrity of received segments using error-detection header field
- **Transport Control Protocol (TCP)**
 - Connection-oriented
 - Reliable (retransmits lost packets)
 - Flow control (sender/receiver speed matching)
 - Congestion control (throttles transmission rate to avoid network congestion)
- **User Datagram Protocol (UDP)**
 - Connection-less
 - Unreliable, no flow control, no congestion control

Multiplexing and demultiplexing

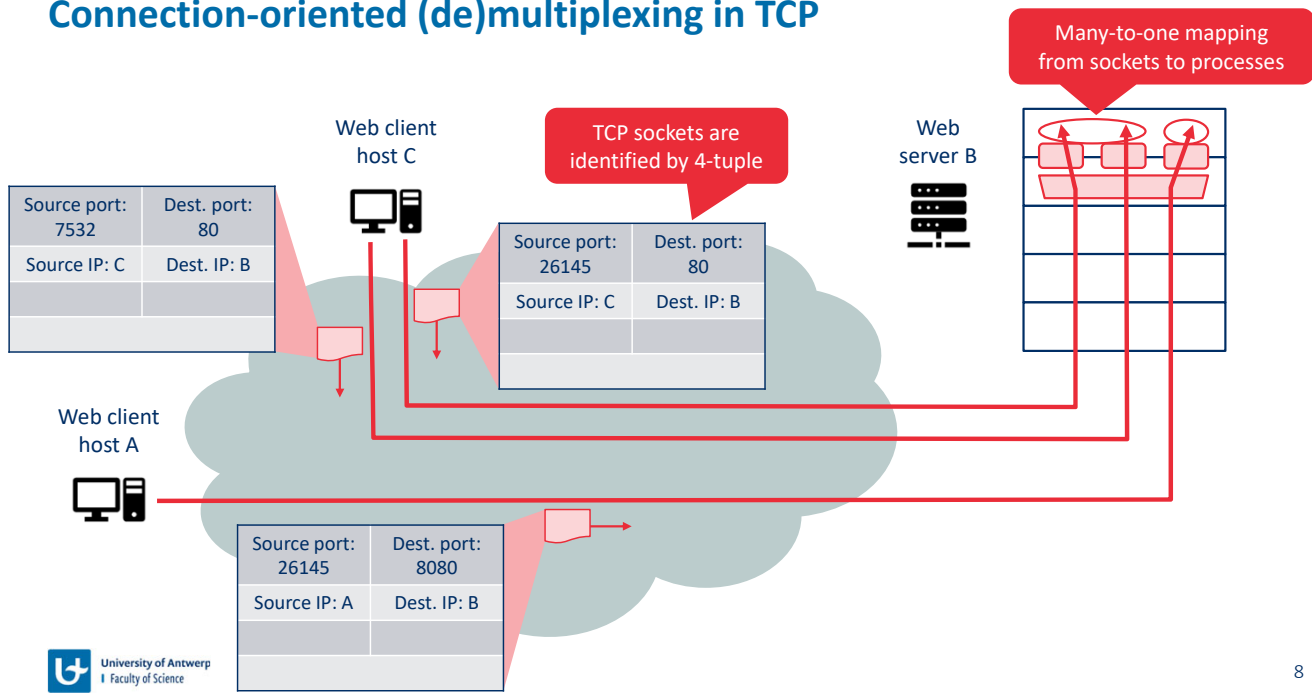


- **Multiplexing:** Encapsulating data from different sockets into segments and passing them to the network layer
- **Demultiplexing:** Identifying the receiving socket based on header fields and directing the data to that socket
- Identification of the correct socket (and thus process) is done based on a **source and destination port number**

Connectionless (de)multiplexing in UDP



Connection-oriented (de)multiplexing in TCP



User Datagram Protocol (UDP)



User Datagram Protocol (UDP) – RFC 768

- UDP is “bare bones”: connection-less, unreliable, no flow or congestion control, ...
- Then why use UDP instead of TCP?
 - Finer application-level control over what data is sent, and when
 - No connection establishment (i.e., less delay before data can be sent)
 - No connection state (i.e., less overhead per connection on the server)
 - Smaller packet header overhead (i.e., 8 bytes for UDP vs. 20 bytes for TCP)

Exercise: Calculate the transport layer overhead for UDP and TCP

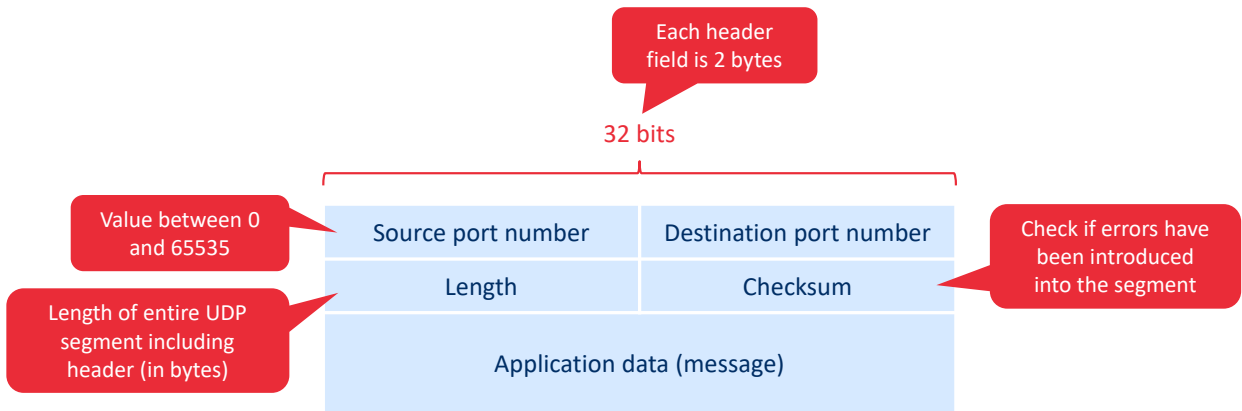


Given a header size of 8 bytes for UDP and 20 bytes for TCP, calculate the percentage overhead of the header for an application-layer message of 16 bytes and 1024 bytes.

Transport protocol used by popular Internet applications

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Secure remote terminal access	SSH	TCP
Web	HTTP, HTTP/3	TCP (HTTP), UDP (HTTP/3)
File transfer	FTP	TCP
Remote file server	NFS	UDP or TCP (v3+)
Streaming multimedia	DASH	TCP
Internet telephony	Typically proprietary	UDP or TCP
Network management	SNMP	Typically UDP
Name translation	DNS	Typically UDP

UDP segment structure



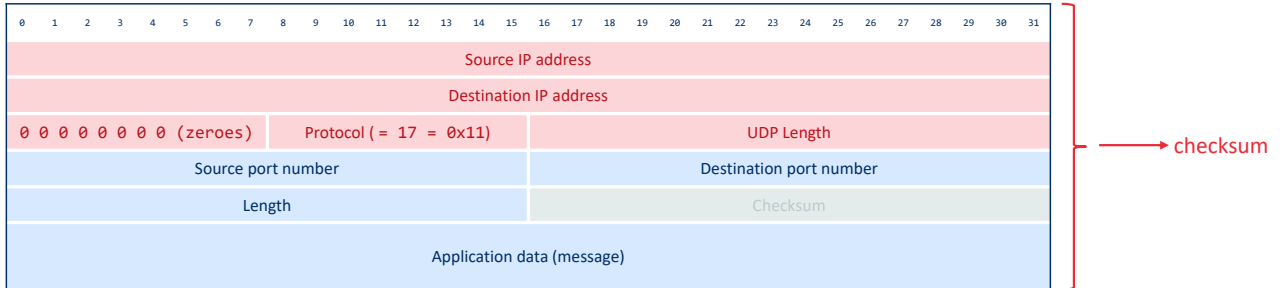
UDP checksum

- Sender

- Calculated by the sender as the 1s complement of the sum of all 16-bit words in the UDP segment (+ IP pseudo header)
- Overflowing bits are wrapped around

- Receiver:

- Add all 16-bit words of the segment together with the checksum
- Result should be 11111111 11111111
- If an error is detected, the packet is usually discarded



UDP checksum example

16-bit input words:

```
01100110 01100000
01010101 01010101
10001111 00001100
```

Sum of words:

```
  01100110 01100000
+ 01010101 01010101
-----
  10111011 10110101
+ 10001111 00001100
-----
  01001010 11000010
```

1s complement:

~ 01001010 11000010

10110101 00111101

Checksum

Limitations of UDP checksum

- Checksum is optional (an all 0s checksum means it hasn't been calculated)
- All 1-bit errors are detected
- Not all 2-bit errors are caught
- Generally, if the sum of all 16-bit words remains constant, the error will not be detected



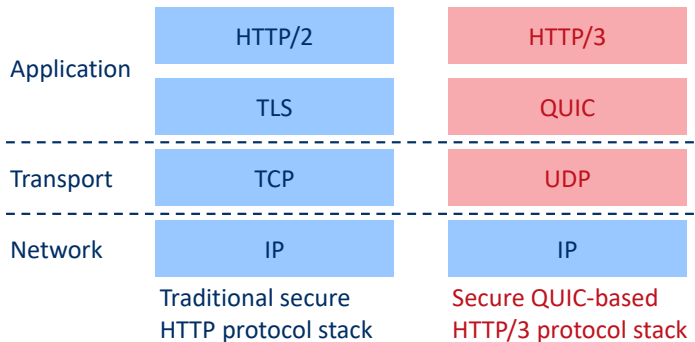
Exercise: Can you think of an example where a 2-bit error is not detected using the UDP checksum?

Original input:

01100110 01100000
01010101 01010101
10001111 00001100

QUIC: A UDP-Based Multiplexed and Secure Transport

- QUIC (originally) stood for Quick UDP Internet Connections (but is now considered the protocol's name)
- Developed and first deployed by Google in 2012
- Standardized since May 2021, as IETF RFC 9000
- About 8,9% of websites use QUIC as of February 2026^[1] (including all Google and Meta webapps)
- Specifically designed to interface with HTTP/3 (IETF RFC 9114, June 2022)



Major features of QUIC and HTTP/3

- **Connection-oriented and secure**

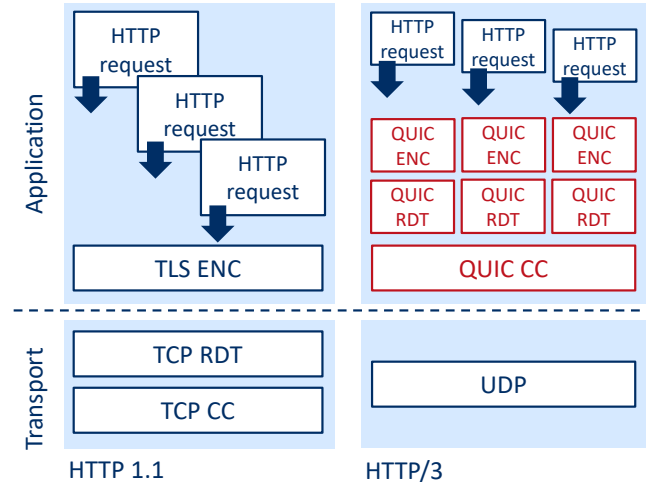
- Single handshake that combines session establishment and authentication/encryption (TCP+TLS separate these)
- Reduces handshake to 1 RTT

- **Streams**

- Multiple application-level streams can be multiplexed over a single QUIC connection
- Quickly add and remove streams
- A stream is a reliable, in-order bi-directional delivery of data between 2 QUIC endpoints (e.g., 1 HTTP/3 object)

- **Reliable, TCP-friendly congestion control**

- Reliability is guaranteed for each QUIC stream separately
- Avoids Head-of-Line (HOL) blocking of multiplexing multiple HTTP responses over a single TCP connection
- Uses TCP NewReno congestion control algorithms



RDT = Reliable Data Transfer

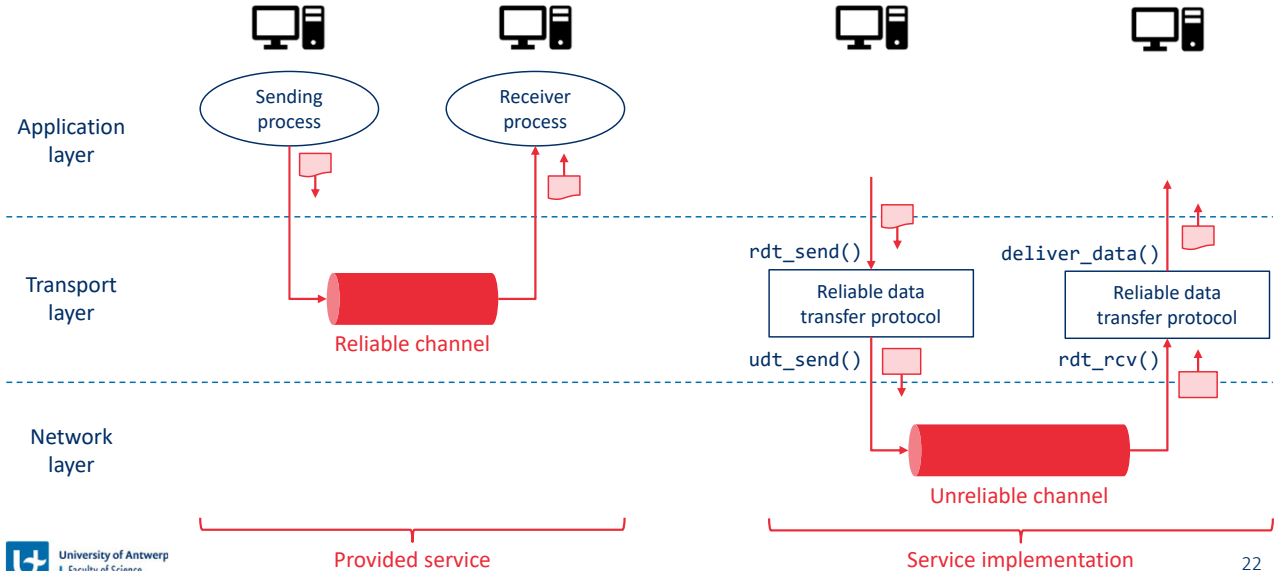
CC = Congestion Control

ENC = Encryption

Reliable data transfer

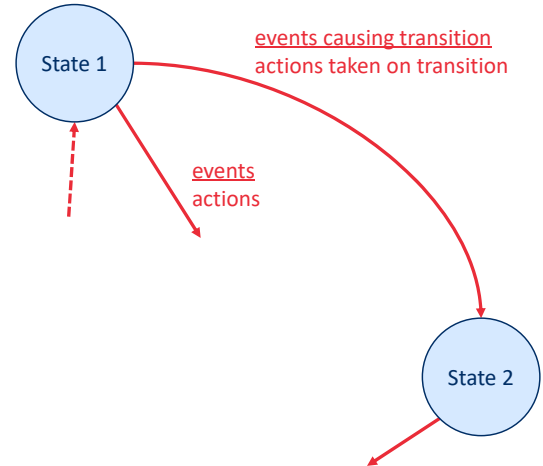


Reliable data transfer over an unreliable channel



Finite state machines (FSM)

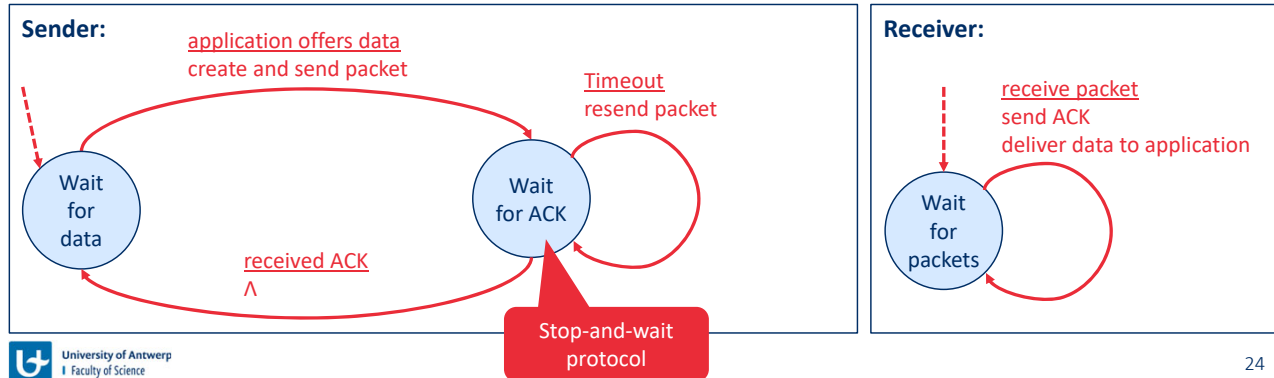
- We will represent protocols using finite state machines
 - Sender and receiver each have their own FSM
 - Start in an “initial state” marked by dashed arrow
 - Events cause a transition from one state to another
- Transition specifies action taken
 - Specified as events/actions
 - Empty event or action is marked by Λ
 - For example: “receive packet/send acknowledgment”



Automatic Repeat reQuest (ARQ) protocols

ARQ is based on 3 fundamental capabilities

- Error detection
- Receiver feedback (e.g., a positive ACK)
- Retransmission by sender (if the received packet did not arrive or had bit errors)



The proposed stop-and-wait protocol has a fundamental flaw

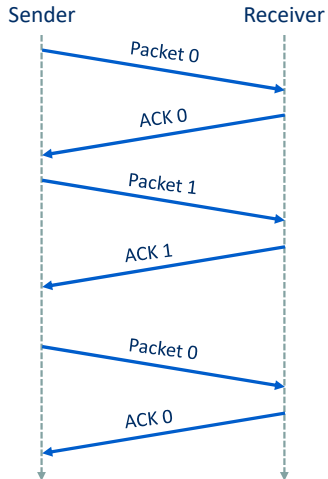
- What happens if an ACK packet gets corrupted or is lost?
- If an ACK is corrupt, the packet can be **resent**
 - This introduces **duplicate packets**
 - The receiver thus needs a way to identify retransmissions
 - This can be done by adding sequence numbers (stop-and-wait only requires a 1-bit sequence number!)
- The sender can detect a lost packet or ACK by using a **timeout**
 - If no response is received after the timeout, either the packet or ACK was lost in transmission
 - Timeout should be at least the duration of a worst-case round-trip-time (RTT)
 - Worst-case RTT is very hard to estimate in most real networks



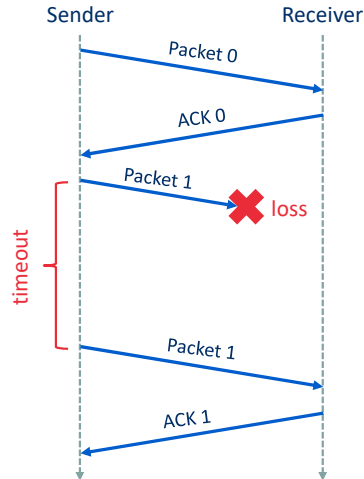
Homework: Develop the FSM for a stop-and-wait sender and receiver with a 1-bit sequence number and timeouts to handle packet loss and corruption of packets and ACKs.

Stop-and-wait alternating-bit protocol

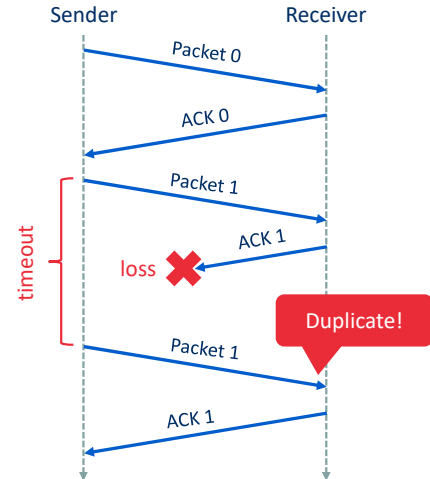
No packet loss:



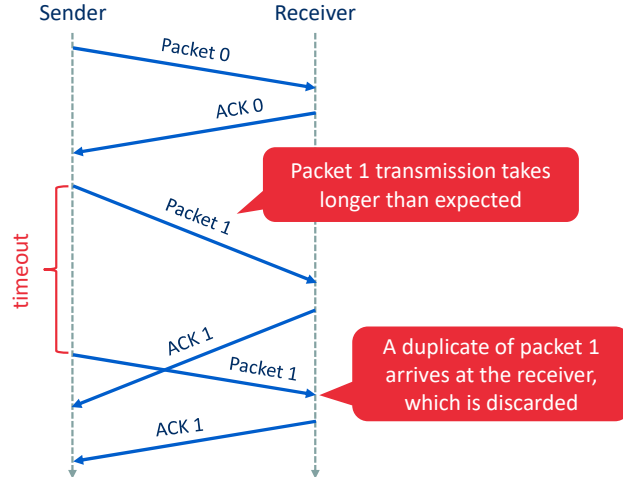
Packet loss:



ACK loss:



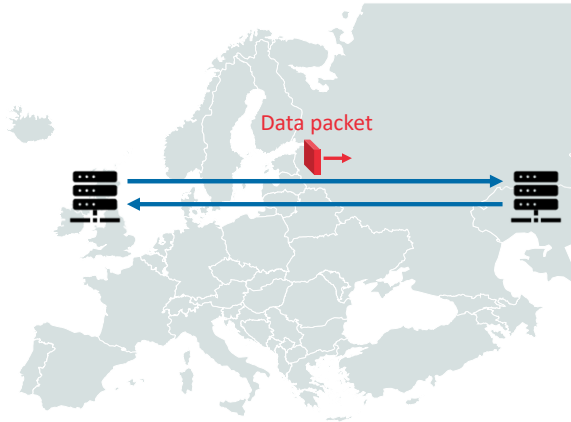
What happens if RTT is higher than timeout?



Stop-and-wait does not offer good performance

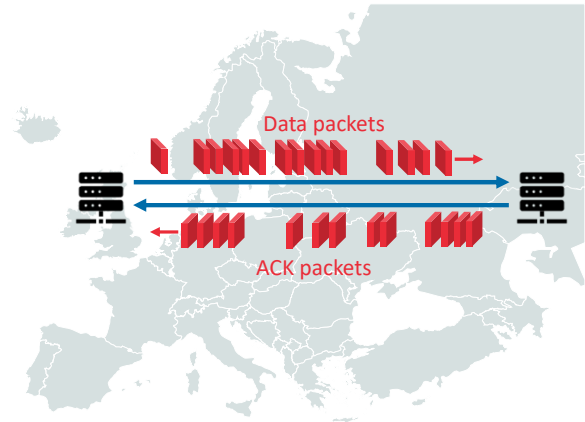
Stop-and-wait

Application layer throughput (goodput) depends solely on the RTT and the packet size



Pipelining

Needed for application goodput to approach link throughput



Exercise: Calculate sender utilization for stop-and-wait protocol



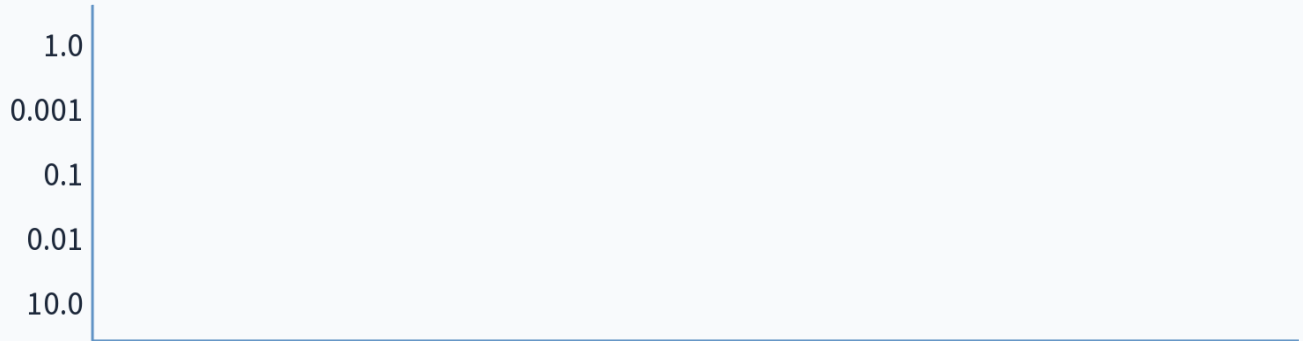
What is the sender utilization (fraction of time the sender is actively transmitting), assuming a 30msec RTT, a 1Gbps link throughput, a packet size (headers + data) of 1000 bytes, and an ACK size of 24 bytes?

Homework: Calculate sender utilization with pipelining



Given the same scenario as in the previous exercise. What would be the sender utilization in case 3-packet pipelining can be used (i.e., 3 unacknowledged packets can be transmitted)?

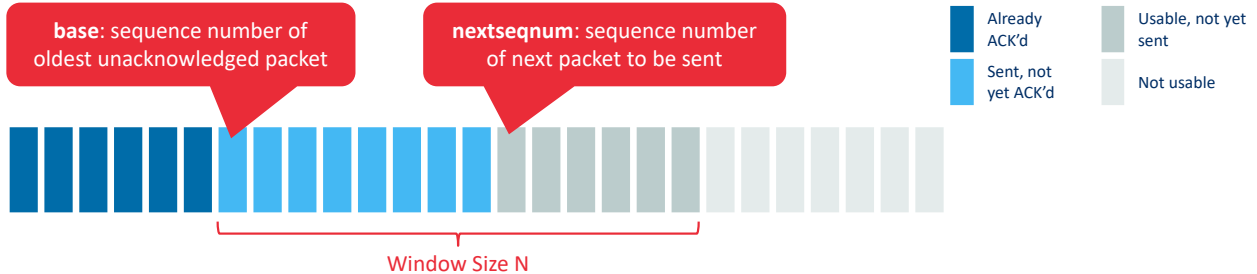
Suppose a packet is 10000 bits long, the channel transmission rate is 10Mbps, and the RTT is 9ms. What is the maximum channel utilization of a stop-and-wait protocol?



Suppose a 10000 bit packet , 10Mbps transmission rate , and 9ms RTT. What is the maximum channel utilization of a pipelined protocol?

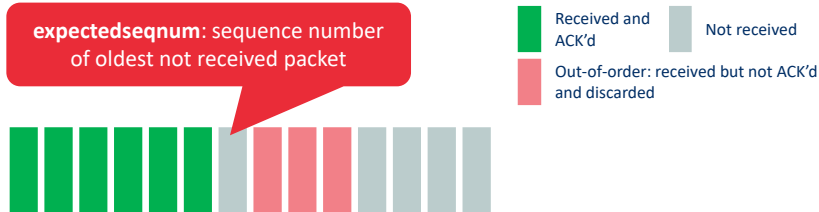
- 1.0 **A**
- 0.001 **B**
- 0.1 **C**
- 0.01 **D**
- 10.0 **E**

Go-Back-N (GBN) sender



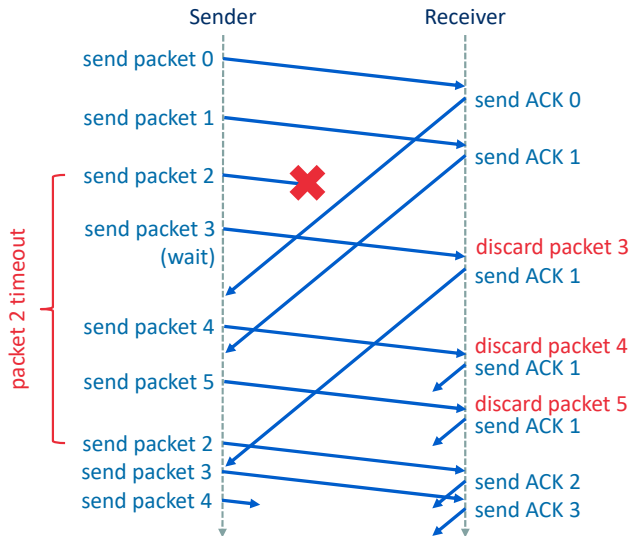
- **Cumulative ACKs:** $ACK(n)$ means all packets with sequence number $\leq n$ were successfully received
 - On receiving $ACK(n)$ move window forward and set $base = n + 1$
- A **timer** is set for oldest unacknowledged packet
 - On timeout, retransmit all unacknowledged packets
- At most N unacknowledged packets can be transmitted at any time

Go-Back-N (GBN) receiver

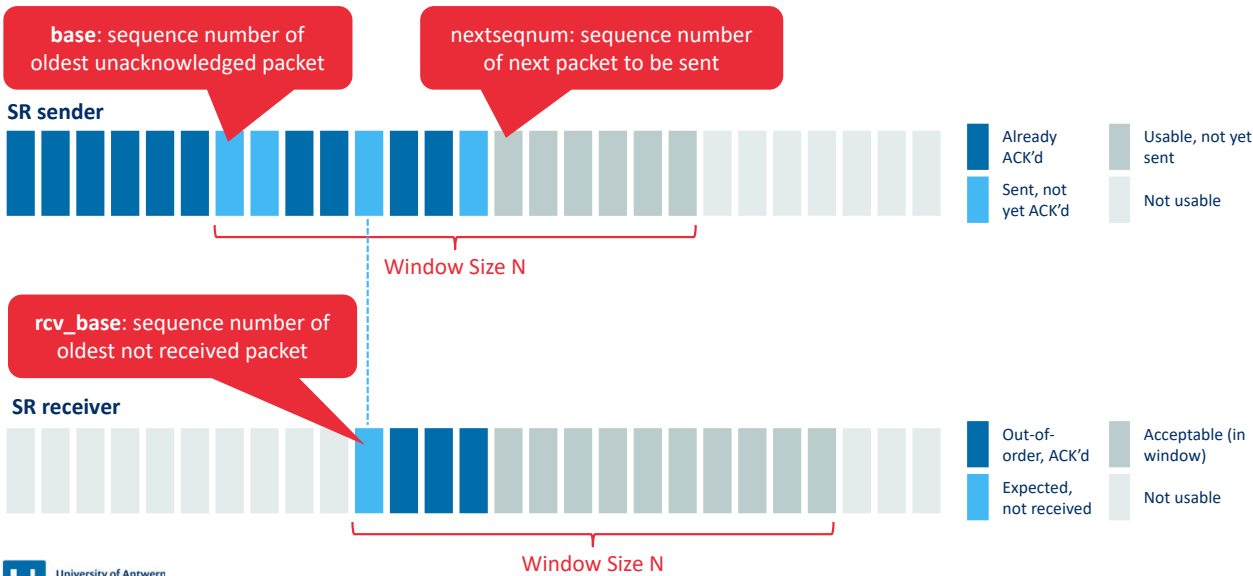


- When the packet with sequence number *expectedseqnum* is received
 - An ACK for that sequence number is sent and *expectedseqnum* is increased by 1
- When an out-of-order packet is received
 - The packet is discarded
 - Another ACK is sent with the highest in-order sequence number

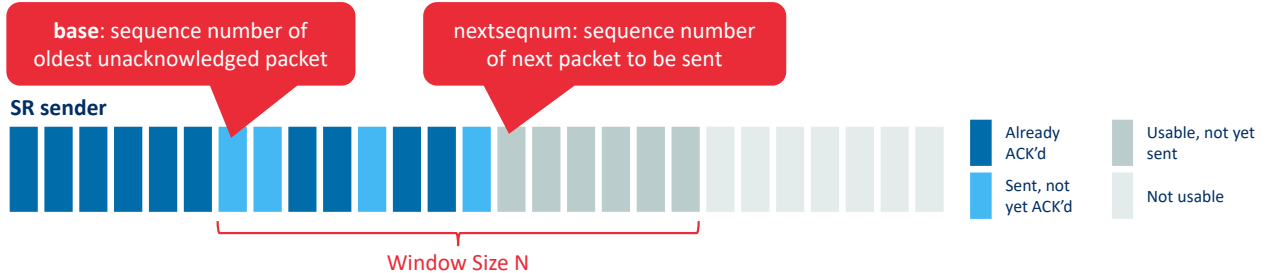
Example: Go-Back-4



Selective Repeat (SR)

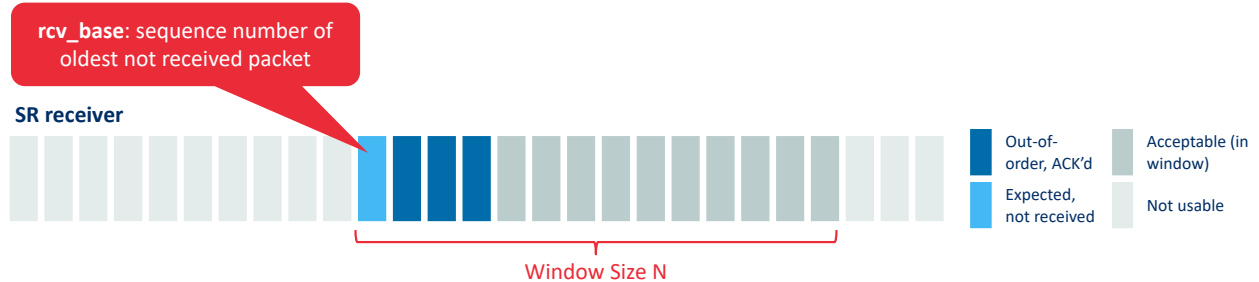


SR sender



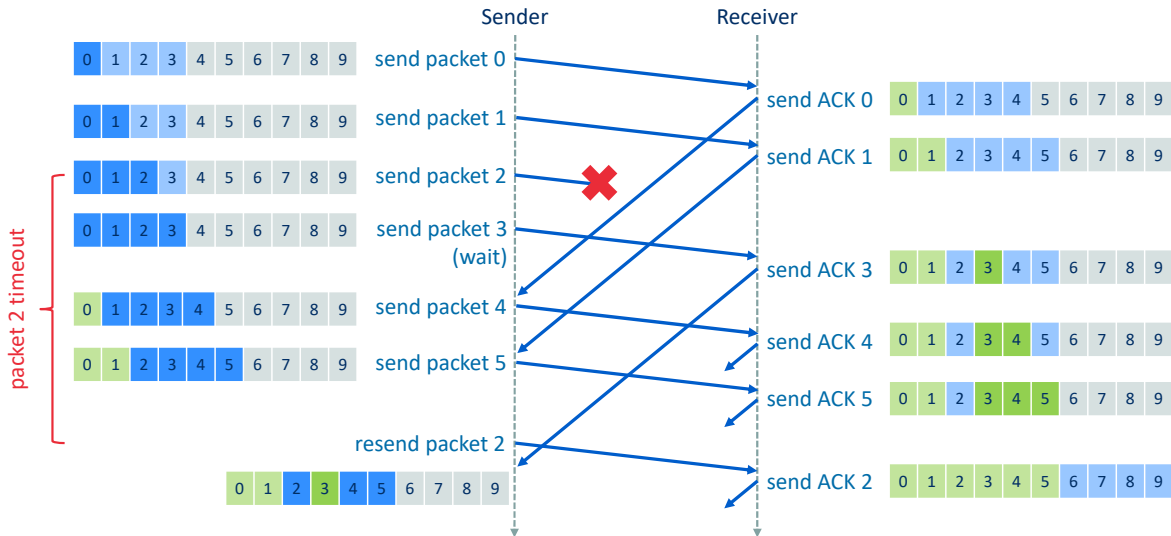
- Data received from the application layer is only transmitted if the next available sequence number falls within the sender's window
- A logical timer is maintained for each packet, since only a single packet is retransmitted on a timeout
- If an ACK is received, the sender marks only that packet as received
 - If the sequence number equals send_base, the window moves to the next unacknowledged packet
 - Untransmitted packets that fall within the new window will then be transmitted

SR receiver



- If a packet arrives within the receive window, a selective ACK is returned and it is buffered
 - If the sequence number equals `rcv_base`, then it and all consecutively numbered buffered packets are delivered to the upper layer, and the receive window is moved forward by the number of delivered packets
- If a packet with sequence number $[\text{rcv_base} - N, \text{rcv_base} - 1]$ is received, an ACK is generated
- Otherwise: Discard the packet

Example: SR with N = 4



How many distinct sequence numbers are needed?

- The window size N must be less than or equal to half the sequence number space
 - This is required in the corner case where all N ACKs of sender window packets are lost
 - Otherwise it might not be possible to differentiate between a new or retransmitted packet

Example

- Window size = 3, and sequence number space = 4
- Scenario 1
 - Packets 0, 1, and 2 are transmitted and received, but all 3 ACKs are lost
 - Senders will retransmit Packet 0 after timeout
- Scenario 2
 - Packets 0, 1, and 2 are transmitted, received, and ACK'd
 - Packet 3 is transmitted but lost after ACK 0
 - Packet 4 (with sequence number 0) is transmitted
- With only 4 sequence numbers, receiver cannot differentiate between Packet 0 retransmission in scenario 1 and Packet 4 transmission in scenario 2 (as both have sequence number 0)

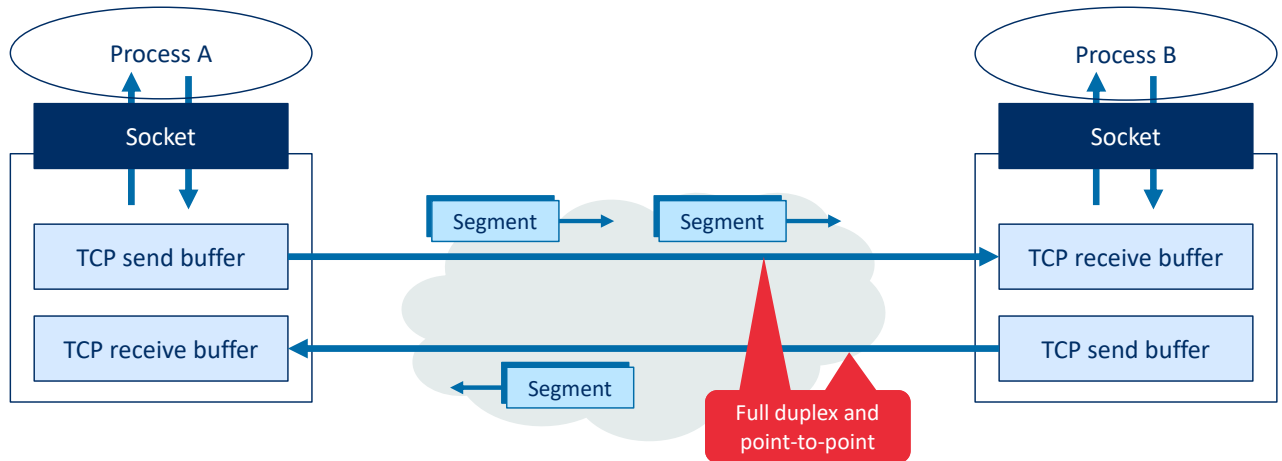
Given a window size $N = 5$ and a selective repeat transmitter and receiver. How many bits should at least be used to store the sequence number?

- 3 **A**
- 4 **B**
- 5 **C**
- 10 **D**
- 11 **E**

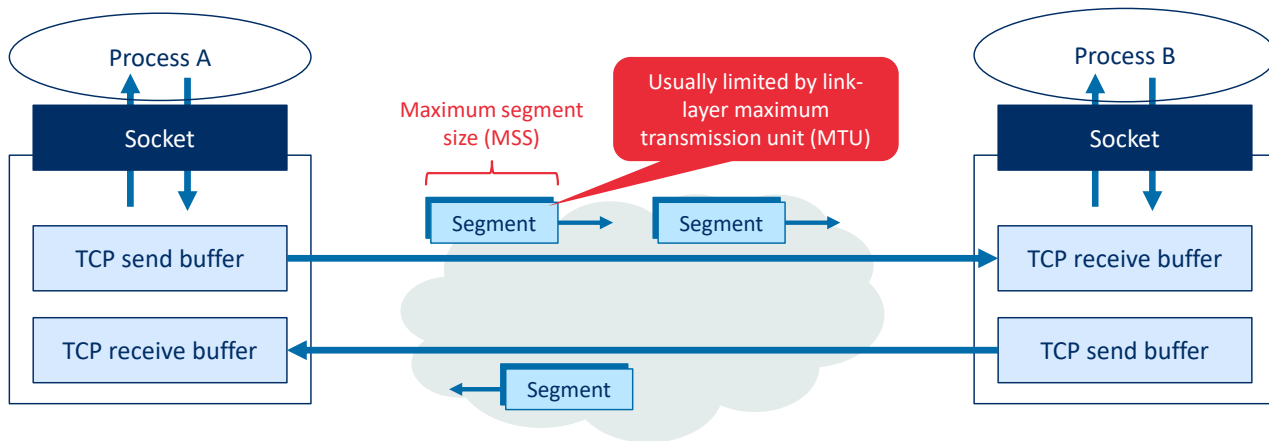
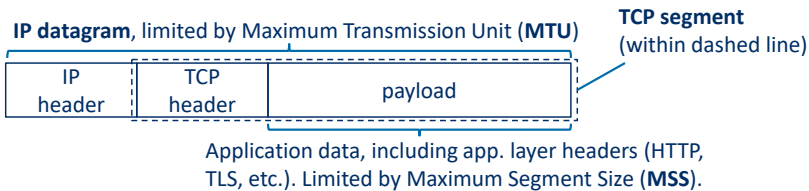
Transport Control Protocol (TCP)



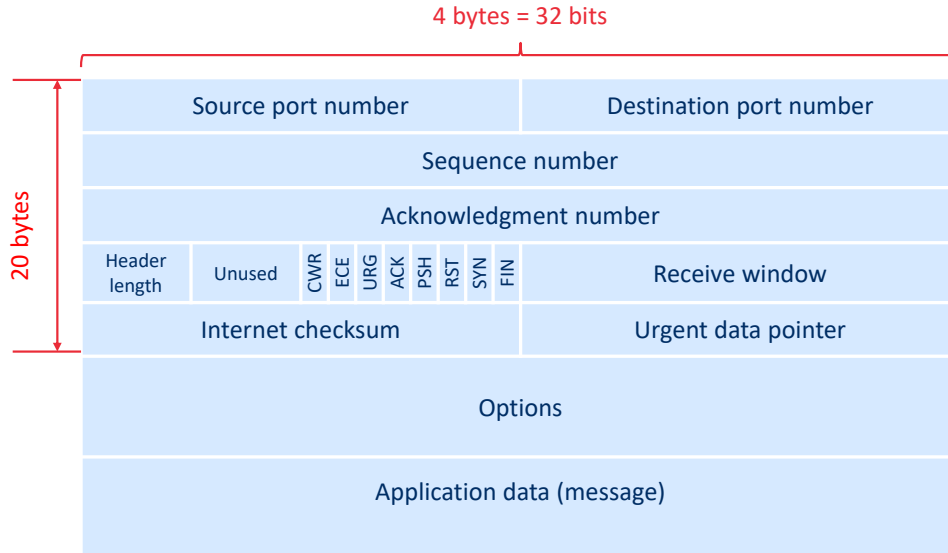
The TCP connection



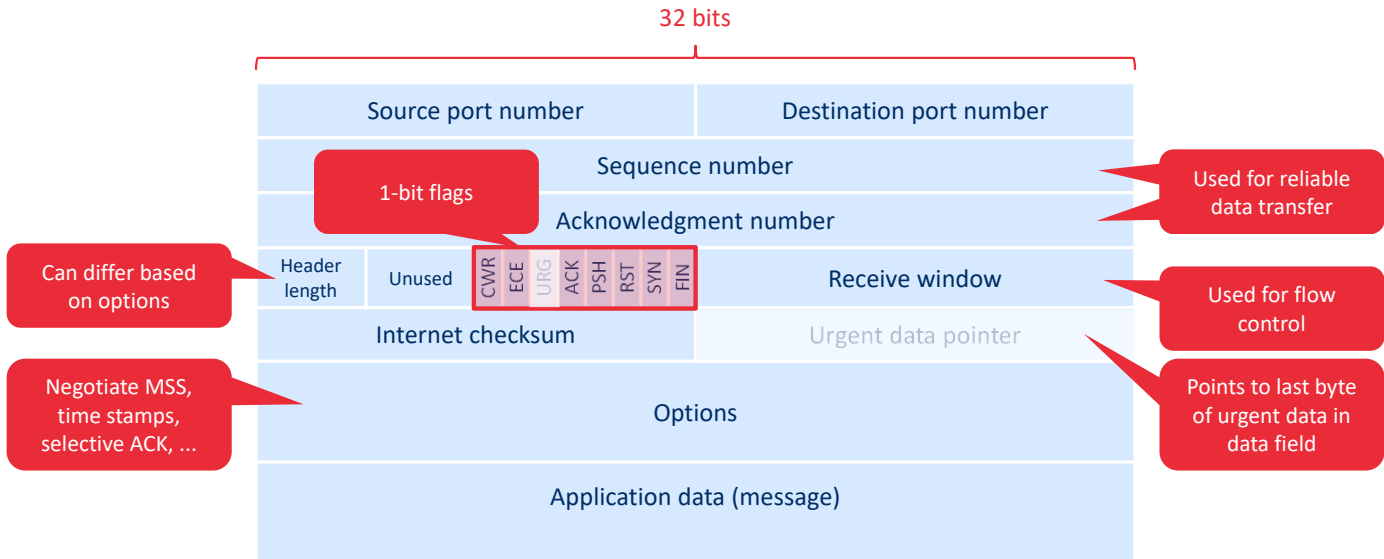
The TCP connection



TCP segment structure

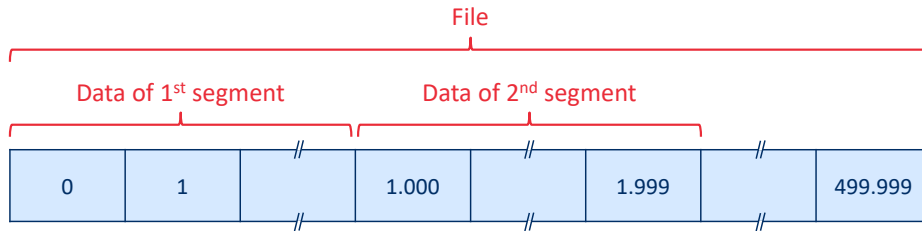


TCP segment structure



Sequence and acknowledgement numbers

- TCP views data as an unstructured, but ordered, stream of bytes
- **Sequence number** of a segment refers to the position of its first byte in the stream and **NOT** the number of the segment itself
- **Acknowledgement number** is the next byte that receiver expects from the sender
 - TCP thus uses **cumulative acknowledgments**
- Example (MSS=1000 bytes, File = 500.000 bytes):



Host A has just successfully received a TCP segment with sequence number 12000. Assuming an MSS of 1000, what acknowledgement number does Host A send back?



(A) 12000

0

(B) 12001

0

(C) 12999

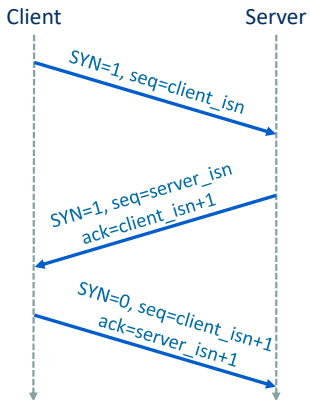
0

(D) 13000

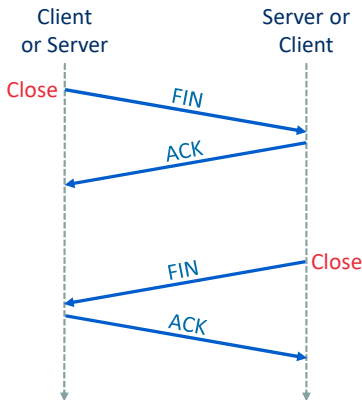
0

Connection management

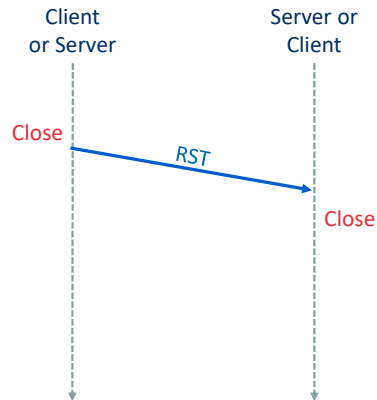
Connection establishment (three-way handshake)



Closing the TCP connection gracefully



Aborting the TCP connection



Estimating the round-trip time (RTT)

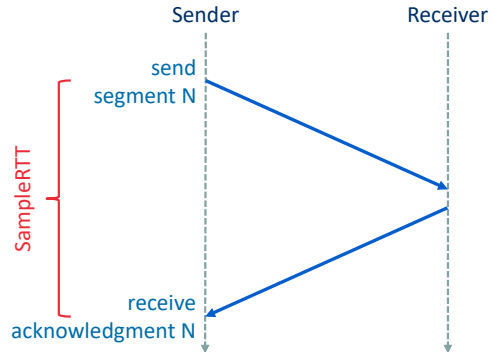
- The TCP retransmit timeout should be larger than the RTT
- How to estimate the RTT in a real network?
 - Measured as the time between sending a segment and receiving the corresponding acknowledgment (**SampleRTT**)
 - Never calculated for segments that are retransmitted
 - To reduce fluctuations, **EstimatedRTT** is an exponential weighted moving average (EWMA) of SampleRTT
 - The variability of the RTT is calculated as **DevRTT**

Recommendation:
 $\alpha = 0.125$

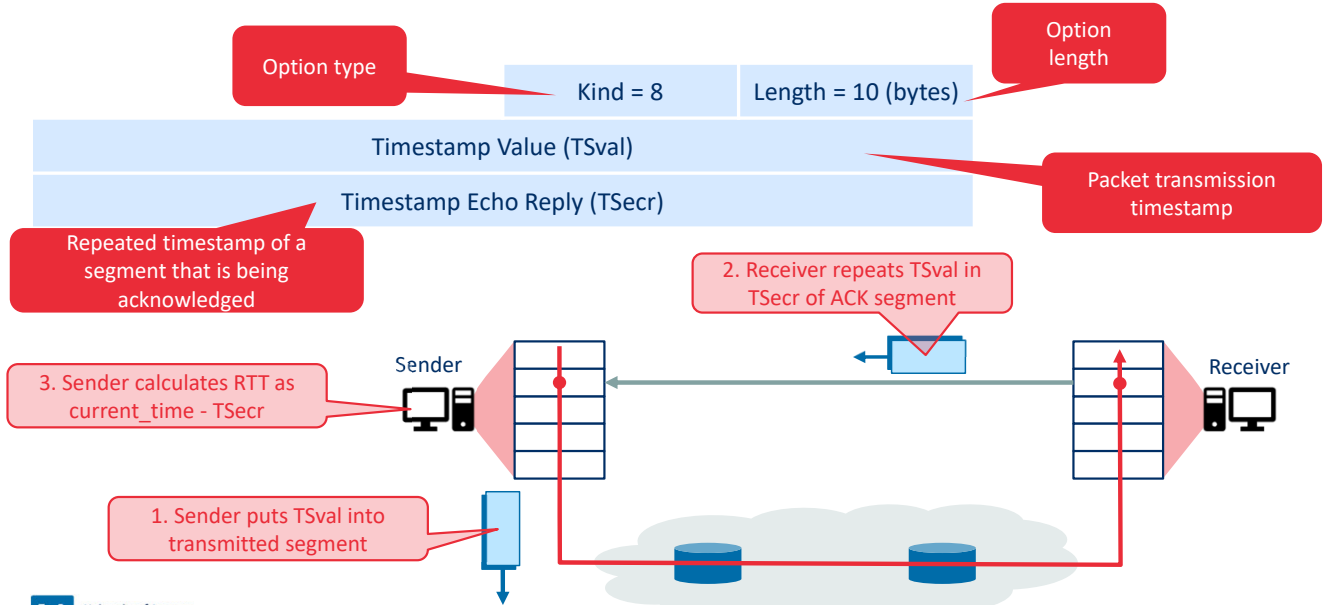
$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

Recommendation:
 $\beta = 0.25$

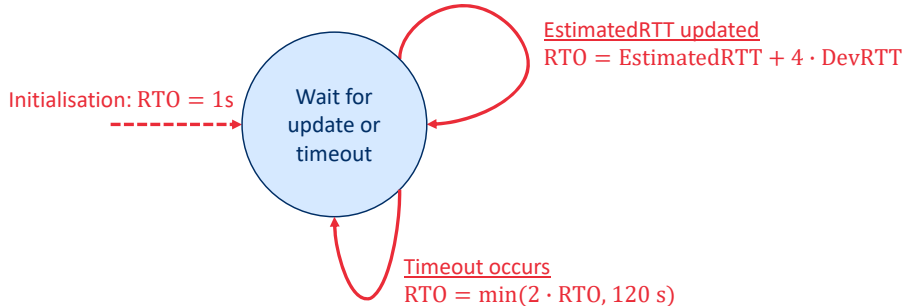


Estimating the round-trip time (RTT) using Timestamp Option

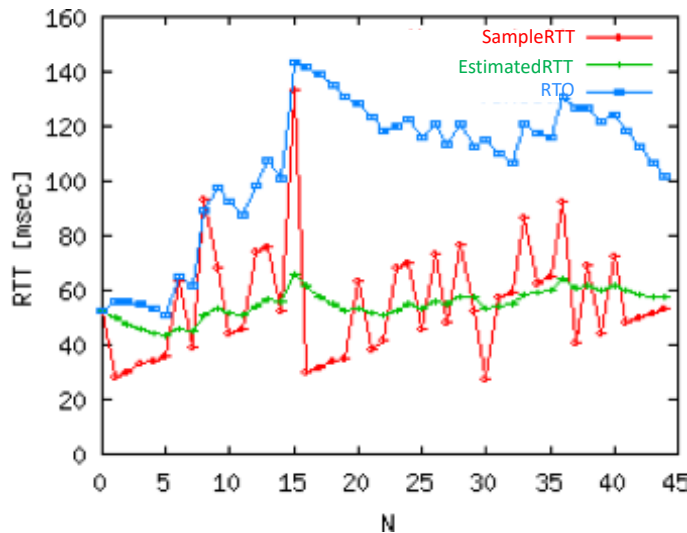


Managing the retransmission timeout (RTO)

- RTO should be close to but larger than EstimatedRTT
 - Margin should be larger if the variability (DevRTT) is large
 - An initial RTO = 1 second is recommended
 - When a timeout occurs, the RTO is immediately doubled, but recalculated when EstimatedRTT is next updated



Example of sampleRTT, estimated RTT, and RTO over time



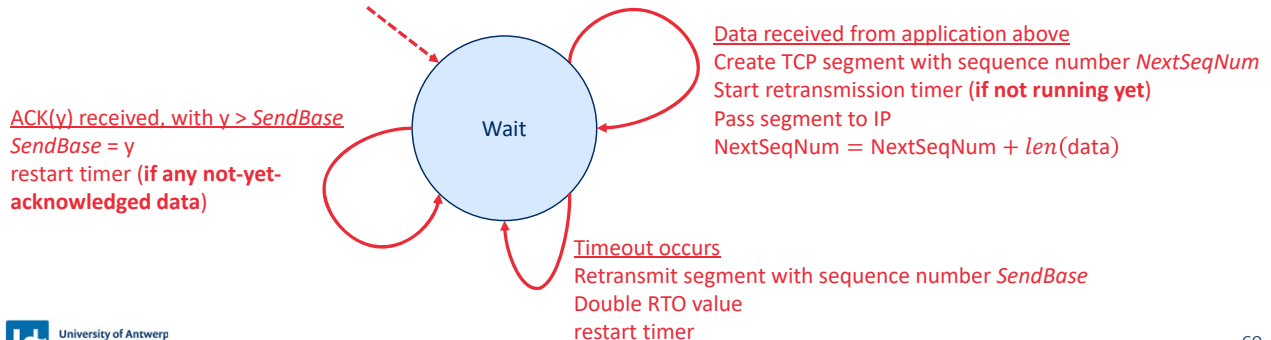
$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

$$\begin{aligned} \text{DevRTT} \\ &= (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}| \end{aligned}$$

$$\text{RTO} = \min(\text{EstimatedRTT} + 4 \cdot \text{DevRTT}, 120\text{s})$$

Retransmission timer management

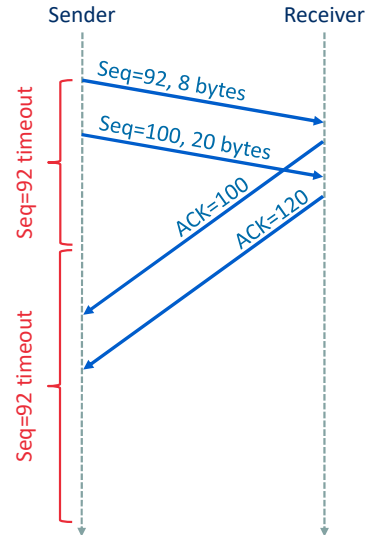
- RFC 6298 recommends the TCP sender keeps a **single** retransmission timer
 - Associated with the oldest unacknowledged segment
 - Restarts when an acknowledgment for this segment is received
 - When the timer expires sender starts retransmission of the oldest unacknowledged segment
- Simplified TCP sender:



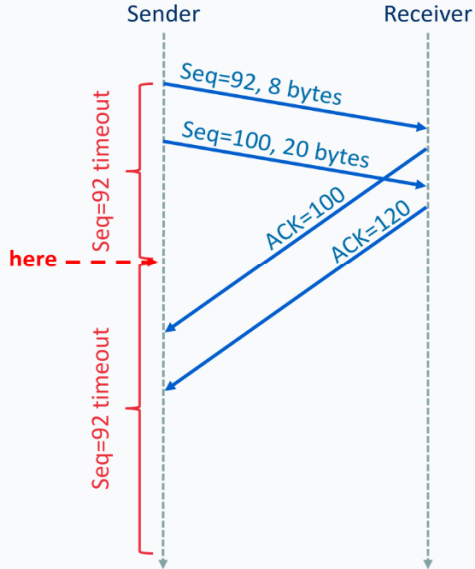
Exercise: Retransmission timeout in TCP

What happens after the first timeout of segment 92?

- A. Only segment 92 will be resent
- B. Segments 92 and 100 will be both be resent
- C. Segment 92 will be resent, and 100 will only be resent after the next timeout
- D. Neither segment will be resent



What happens after the first timeout of segment 92?



Only segment 92 will be resent

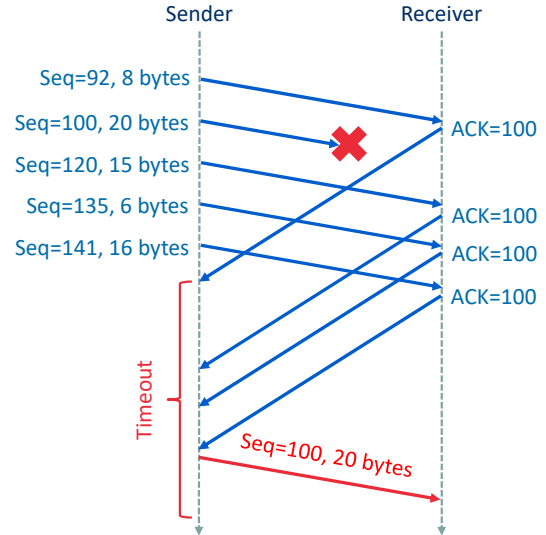
Segments 92 and 100 will both be resent

Segment 92 will be resent, and 100 will only be resent after the next timeout

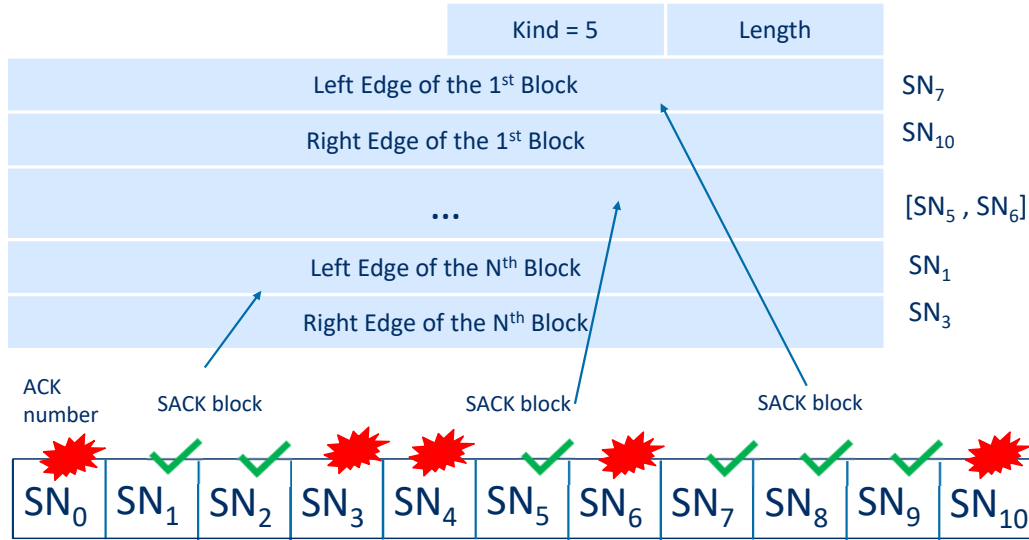
Neither of the segments will be resent

Fast retransmission

- The RTO value can be relatively long, especially for high-speed (e.g., Gigabit Ethernet) and high-RTT (e.g., satellites) networks.
- Lost packets can be detected earlier by counting **duplicate ACKs**, whose acknowledge number corresponds to already acknowledged segment.
- After **3 duplicate ACKs**, the oldest non-acknowledged segment is retransmitted.
- If a retransmission timeout occurs, it is an indication of significant problems or changes in the network.



TCP Selective Acknowledgement (SACK) option



Is TCP's error recovery Go-Back-N (GBN) or Selective Repeat (SR)?

Go-Back-N

Selective Repeat

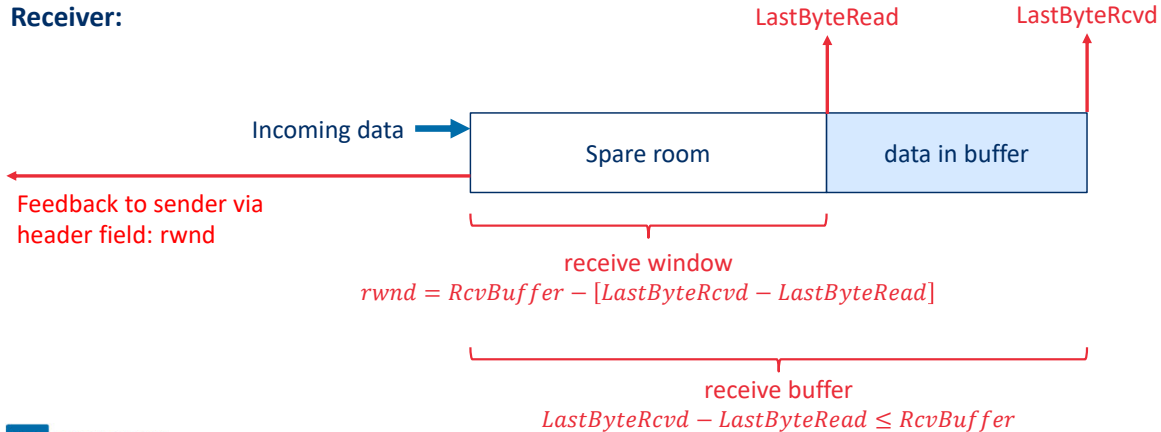
A little of both

Neither

Flow control – Receiver

Ensures that the sender does not send data faster than the receiver application reads it

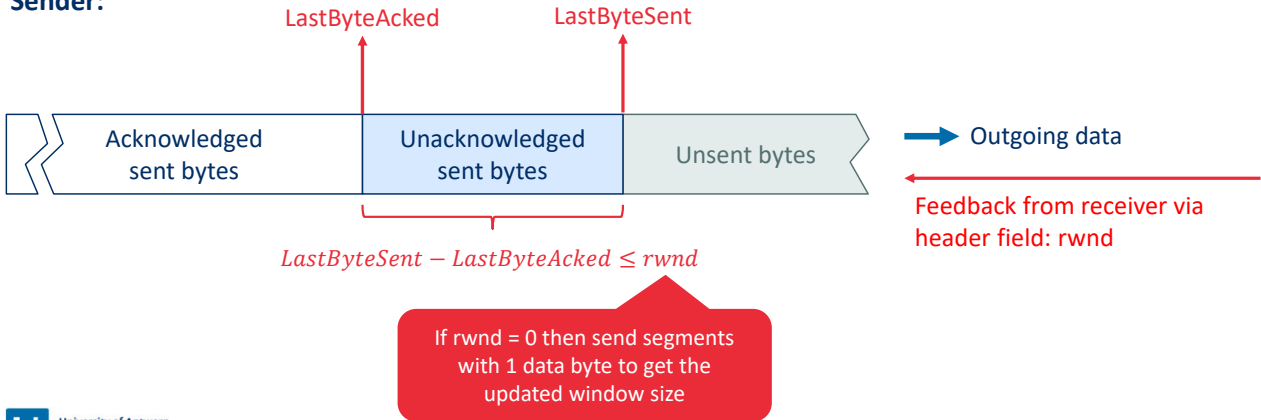
Receiver:



Flow control – Sender

Ensures that the sender does not send data faster than the receiver application reads it

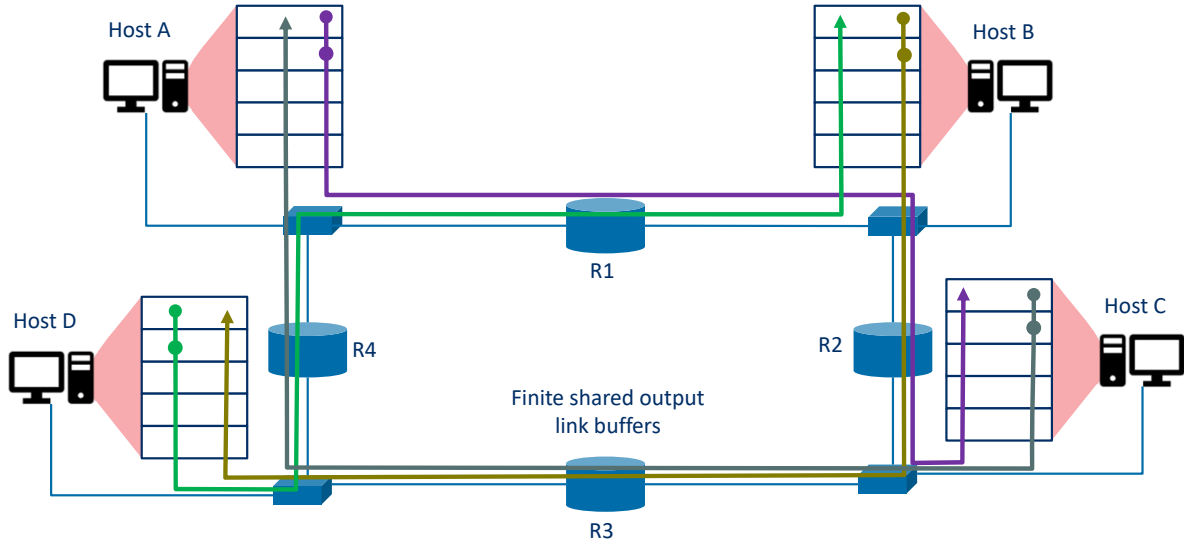
Sender:



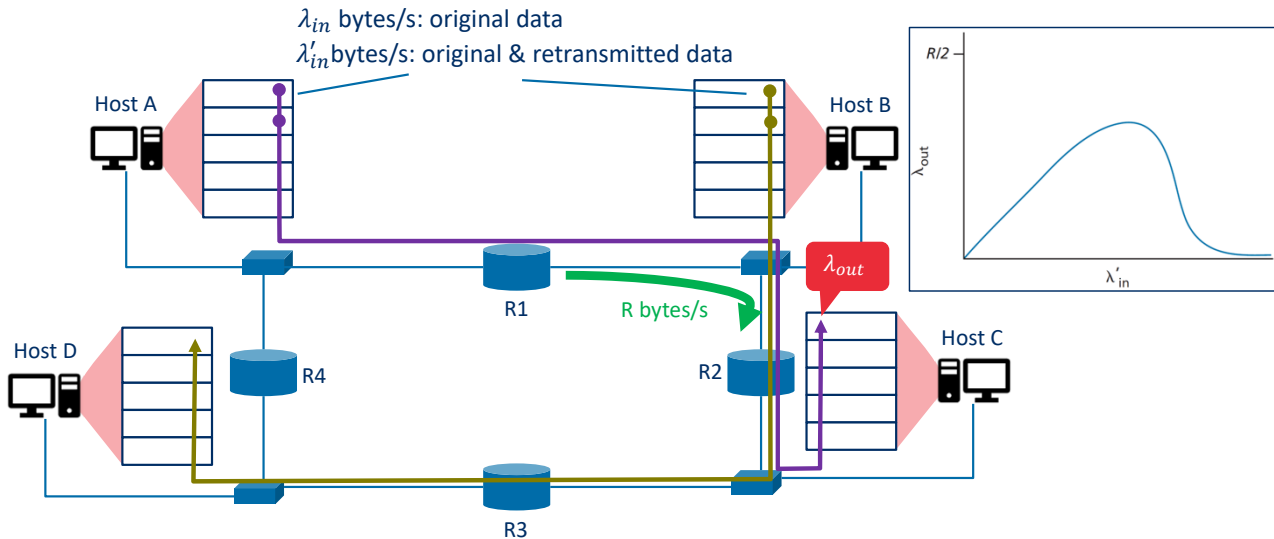
TCP congestion control



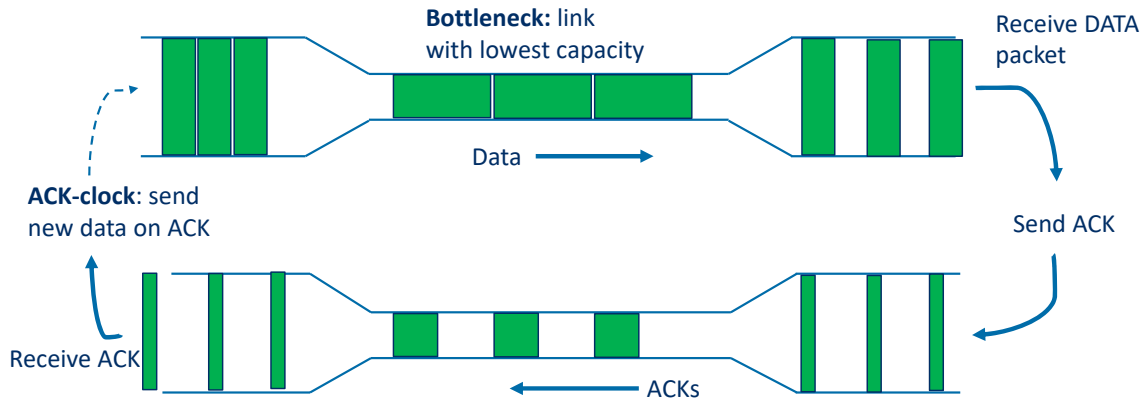
Understanding congestion



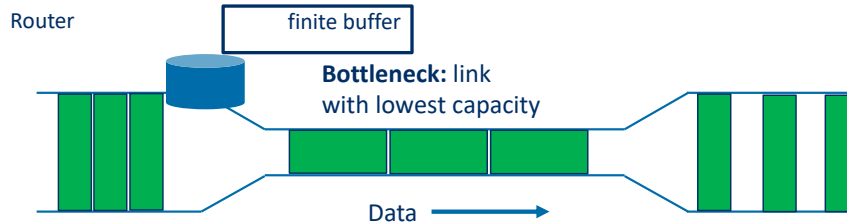
Understanding congestion



Understanding congestion: bottleneck



Optimal pipelining for single data flow



Optimal amount of data “in flight”, i.e., sent but not acknowledged, is determined by:

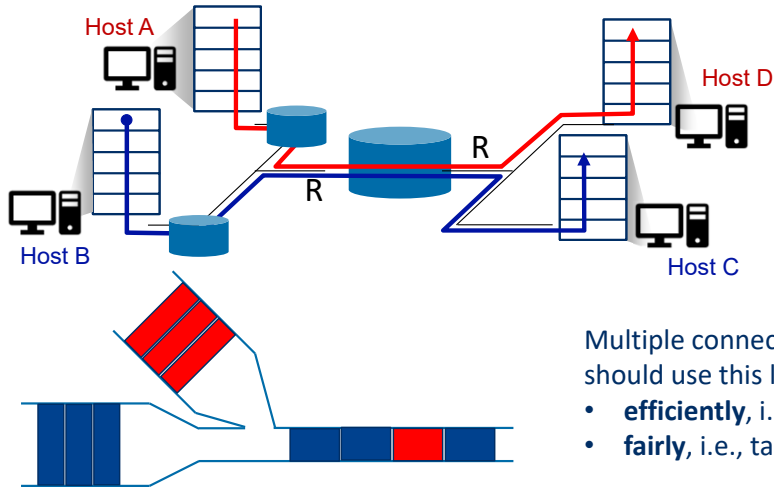
- Bottleneck link capacity C bytes/s;
- Round-trip time RTT s.

To fully fill the bottleneck link with packets “back-to-back” **without queueing**, the sender should pipeline $C \cdot RTT$ bytes. This number is called **bandwidth-delay product (BDP)**.

When the sender starts transmitting more than BDP data, the queueing will start happening. When the router’s buffer will become full, this router will start dropping packets.*

*In practice, it will start dropping packets earlier to avoid bursty drops.

Fairness for multiple data flows



Multiple connections sharing the same bottleneck link should use this link:

- **efficiently**, i.e., with maximum utilization;
- **fairly**, i.e., take equal shares of its capacity.

Summary: goals of congestion control

- Throttle (i.e., decrease transmission rate) senders in the face of network congestion
- Efficiently utilize the capacity of the bottleneck to maximize throughput
- Fairly share the bottleneck link between multiple connections

Two main approaches to congestion control

End-to-end congestion control

- Presence of congestion is inferred by end systems based on observed network behaviour
- Approach used by TCP
 - TCP packet loss is used to identify congestion*
 - TCP decreases window size accordingly

*In practice, packet loss can be caused not only by network congestion, but due to different other reasons. Notably, in wireless networks (such as Wi-Fi) many packet losses happen due to high variability of the wireless medium and interference.

Network-assisted congestion control

- Routers provide explicit feedback to the sender and/or receiver regarding congestion state
- Can be a single bit or more detailed feedback
- Revisited in relatively new standard L4S (RFC 9330) actively pushed by Nokia.
- Used by IBM SNA, DECnet, ATM, ...
- Supported by TCP, but many “middleboxes” (routers, firewalls, etc.) do not support it.

TCP congestion control

- TCP limits the rate at which it sends traffic as a function of perceived network congestion

- This approach raises several questions

Sender uses congestion window (cwnd)
$$LastByteSent - LastByteAcked \leq \min(cwnd, rwnd)$$

- How does a TCP sender limit its sending rate?

- How does a TCP sender perceive that there is congestion?

Timeout (severe problem) or three duplicate ACKs (congestion just started)

- What algorithm should the sender use to adapt its send rate as a function of congestion?

- Consists of 3 parts

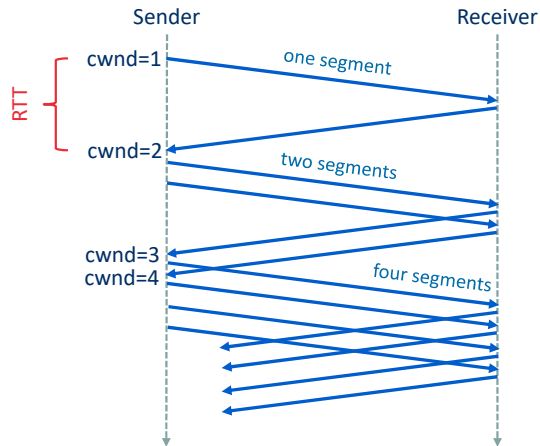
- Slow start
 - Congestion avoidance
 - Fast retransmit & fast recovery

Decrease cwnd when segments are lost. Decrease stronger in case of timeout.

Increase cwnd when ACKs are received

State 1: Slow start

- *cwnd* is initialized to *init_cwnd* MSS and increased by 1 MSS on every acknowledged MSS
 - “vanilla” slow start: *init_cwnd* = 1 MSS
 - RFC 5681: *init_cwnd* ~ 4000 bytes (3 MSS for MSS = 1460 bytes)
 - RFC 6928 (Linux default): *init_cwnd* = 10 MSS
- When *cwnd* $\geq$ *ssthresh* (slow start threshold), TCP transitions into **congestion avoidance mode**
- Initial value of *ssthresh* is Infinity (= no threshold)



Given a slow start congestion control mechanism (default Linux implementation) with MSS = 500 bytes, RTT of 200 ms, and a link capacity of 10 Mbps, what is the initial sending rate?

500 bps

2.5 kbps

25 kbps

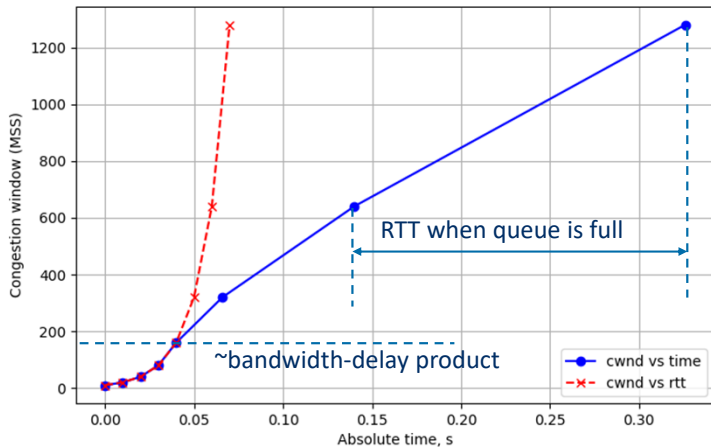
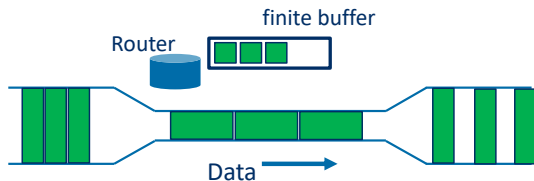
200 kbps

10 Mbps

State 1: Slow start

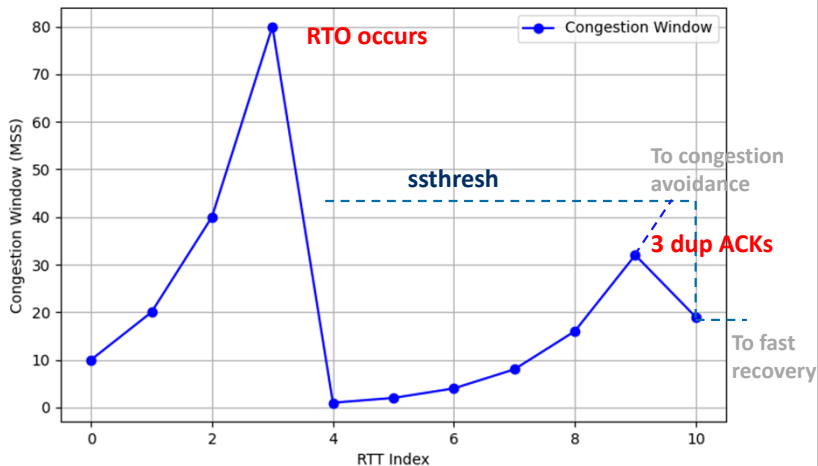
cwnd is increased twice in every RTT.

However, in absolute time scale, RTT starts increasing when *cwnd* reaches bandwidth-delay product.



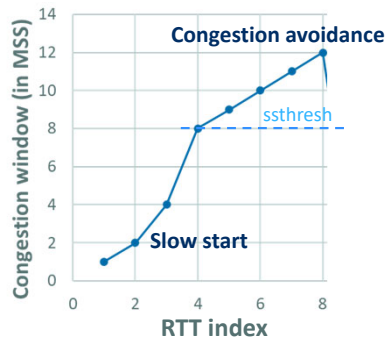
State 1: Slow start

- When a **timeout** occurs
 - ssthresh is set to $cwnd / 2$
 - cwnd is reset to 1 MSS
- When 3 duplicate ACKs are received TCP transitions into **fast recovery** state
 - ssthresh is set to $cwnd / 2$
 - cwnd is set to $(cwnd / 2 + 3 \text{ MSS})$



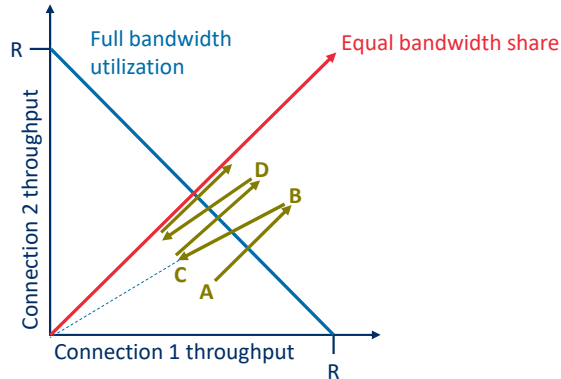
State 2: Congestion avoidance [TCP Reno, Additive Increase Multiplicative Decrease (AIMD)]

- Starts when $cwnd$ reaches $ssthresh$ (half of $cwnd$ value when congestion was last encountered)
- TCP only increases $cwnd$ by 1 MSS every RTT (instead of doubling it as in slow start mode)
 - Common approach: When new ACK arrives, $cwnd = cwnd + 1/cwnd$. The idea is to increase $cwnd$ by 1 after $cwnd$ ACKs are received (i.e., after ~ 1 RTT).
 - If $cwnd$ is measured in bytes, the formula transforms into $cwnd_bytes = cwnd_bytes + MSS^2/cwnd_bytes$
- TCP goes to:
 - slow start state when a timeout occurs ($ssthresh = \frac{cwnd}{2}$; $cwnd = 1$);
 - fast recovery after 3 duplicate ACKs ($ssthresh = \frac{cwnd}{2}$; $cwnd = ssthresh + 3 MSS$).



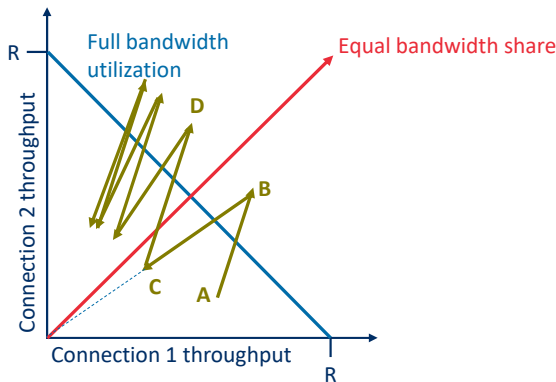
Fairness of TCP Reno's AIMD approach

- A congestion control mechanism is **fair** if K connections sharing a bottleneck link with transmission rate R bps, each achieve an average transmission rate of approximately R/K .
- The TCP Reno's AIMD algorithm has been shown to be fair in certain cases.
- Let's consider an example of a link with transmission rate R shared by 2 TCP connections **with equal RTTs**



RTT unfairness of TCP Reno's AIMD approach

- A congestion control mechanism is **fair** if K connections sharing a bottleneck link with transmission rate R bps, each achieve an average transmission rate of approximately R/K .
- The TCP Reno's AIMD algorithm relies on ACK clocks. The connection with smaller RTT will increase CWND faster. Besides, throughput formula (approx. cwnd/RTT) has RTT in denominator.
- Let's consider an example of a link with transmission rate R shared by 2 TCP connections **with non-equal RTTs** ($\text{RTT}_1 > \text{RTT}_2$).



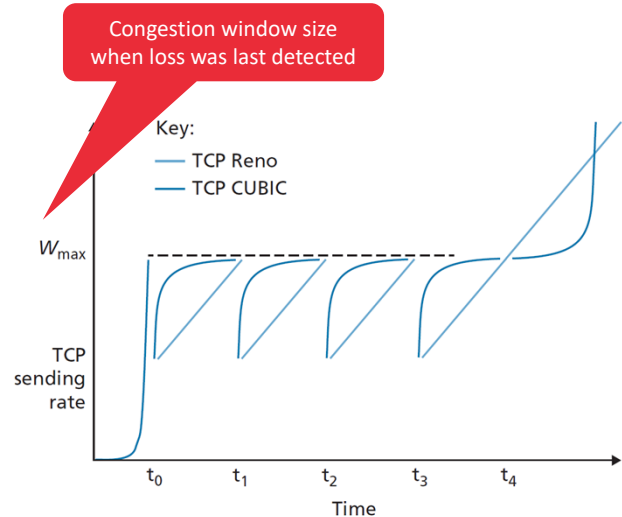
State 2: Congestion avoidance [TCP CUBIC, default in many OS]

TCP Reno

- Additive-increase, multiplicative-decrease (AIMD)
- Slowly increases cwnd after packet loss
- Window increases by $\frac{MSS^2}{cwnd}$ bytes on every ACK

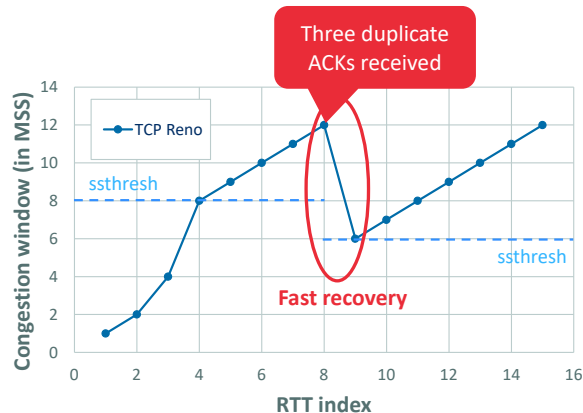
TCP CUBIC

- Quickly ramps up sending rate to get close to pre-loss sending rate
- Probes cautiously afterwards
- Window increase function depends on the time elapsed since congestion event, not on ACKs arrival rate => better RTT-fairness.



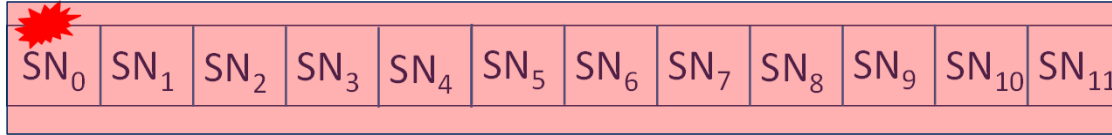
State 3: Fast recovery

- Fast recovery starts when 3 duplicate ACKs are received during slow start or congestion avoidance
 - $ssthresh$ is set to $cwnd / 2$
 - $cwnd = ssthresh + 3 \cdot MSS$
- On each next duplicate ACK $cwnd$ is increased by 1 MSS (window inflation)
- When a **timeout** occurs, TCP goes to slow start (setting $cwnd$ to 1 MSS and $ssthresh$ to $cwnd / 2$)
- When an **ACK arrives** for the missing segment, TCP goes back to congestion avoidance. $cwnd$ is reset to $ssthresh$ (window deflation)



State 3: Fast recovery

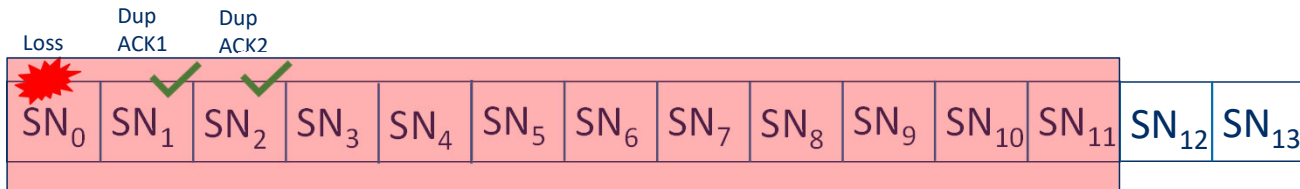
Loss



TCP segment with sequence number SN₀ is lost.

Current congestion window size $cwnd = 12$ MSS

State 3: Fast recovery

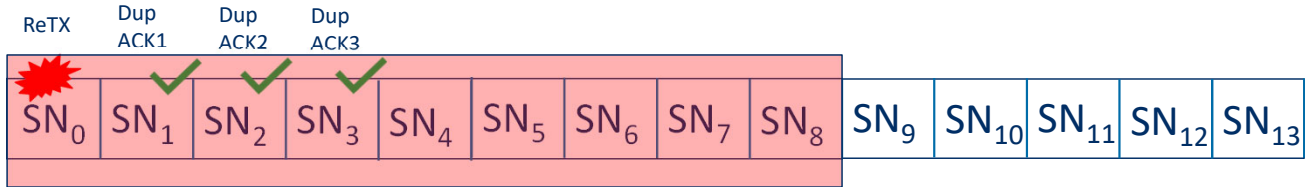


First two duplicate ACKs arrived.

ACKs confirm that some data segments have been received => two data segments has been removed from pipe.
TCP transmits 2 segments to substitute these segments (limited transmit) [RFC 3042].

Segments with sequence numbers SN₁₂ and SN₁₃ are transmitted.

State 3: Fast recovery

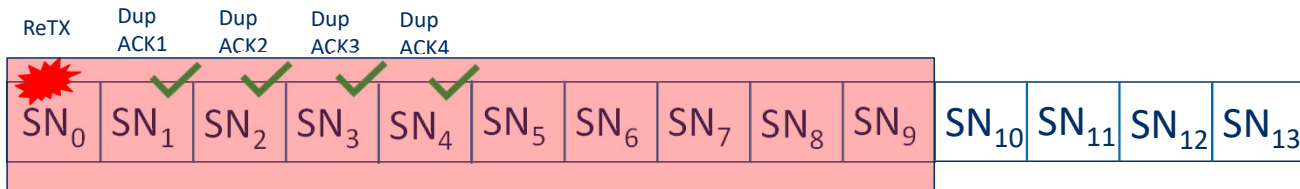


Third duplicate ACK arrived.

Fast retransmit is performed. TCP retransmits segment with sequence number SN₀.

Fast recovery starts. Slow start threshold is set to $ssthresh = 6 \text{ MSS}$, congestion window is set to $cwnd = ssthresh + 3 \text{ MSS} = 9 \text{ MSS}$.

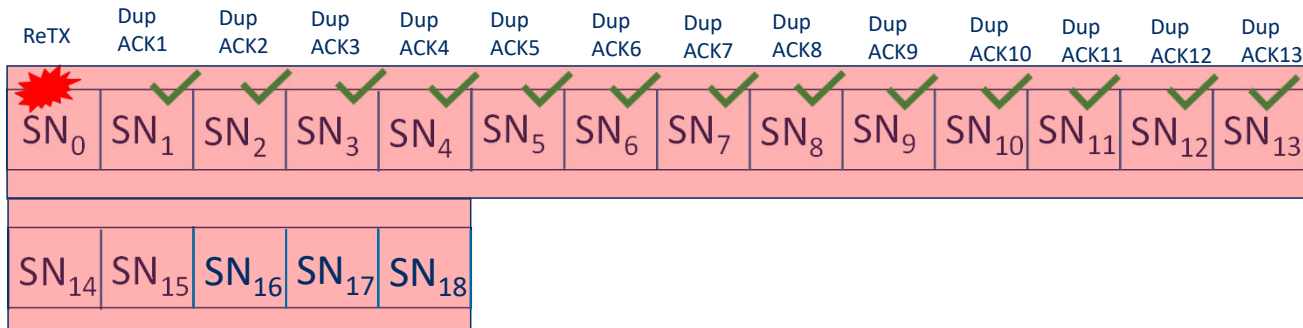
State 3: Fast recovery



Fourth duplicate ACK arrived.

One more segment left the pipe => TCP “inflates” window to compensate (TCP conservation law). Since segment with SN₉ has been already sent, nothing will be sent.

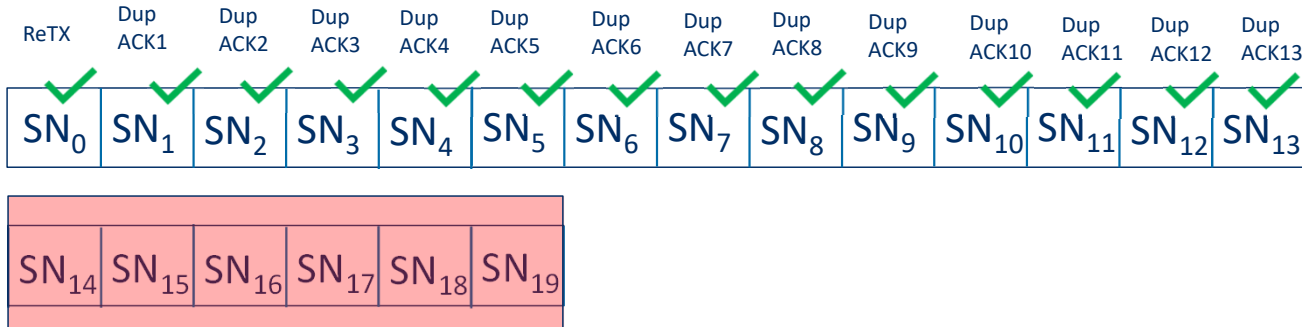
State 3: Fast recovery



13th duplicate ACK arrived.

TCP continue “inflating” window. New segments with sequence numbers SN₁₄, SN₁₅, SN₁₆, SN₁₇, and SN₁₈ are sent.

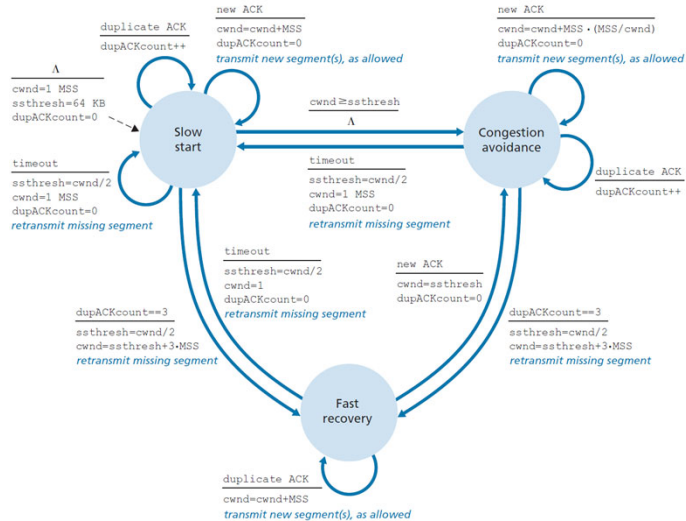
State 3: Fast recovery



Acknowledgement for TCP segment with sequence number SN₀ finally arrives. Fast recovery is finished.

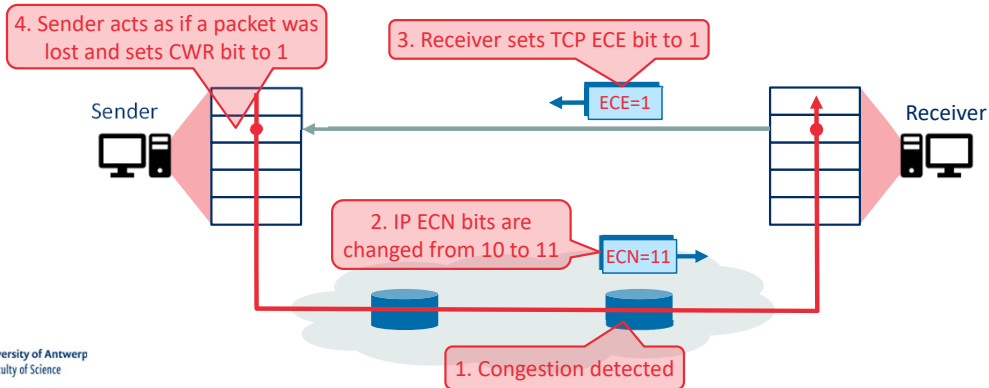
TCP deflates congestion window. The new value is: $cwnd = ssthresh = 6 \text{ MSS}$. Segment with sequence number SN₁₉ is sent. TCP congestion avoidance start.

Summary: FSM description of TCP congestion control

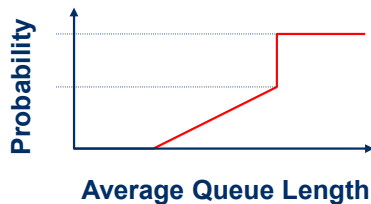
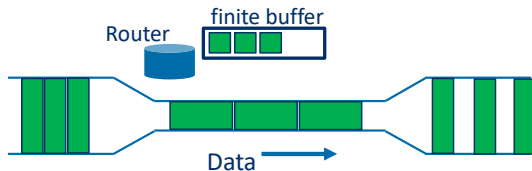


Explicit congestion notification (ECN) [RFC 3168]

- Uses 2 bits in the **IP** header
 - ECN-capable transport (ECT) bit is set to 1 if the sender and receiver support ECN
 - Congestion Experienced (CE) bit is set to 1 if an intermediary router experiences congestion
- What qualifies as congestion is left to the equipment vendor and/or network operator to define
- If the CE bit is 1 on a received TCP packet, the receiver will set the TCP ECE bit to 1 in its ACK
- If the sender receives an ACK with ECE=1 then it acts as if a packet was lost and sets CWR=1 in next segment



Active queue management



- When the buffer of the router becomes full, it is already too late to start dropping the packets.
 - long delay because of waiting full queue of packets;
 - butch-losses because multiple packets arrive to the fully occupied buffer
- Active queue management addresses this issue. The idea is to drop packets earlier to throttle TCP senders
- Example: *Random Early Detection (RED)* is designed to drop packets randomly. The probability of dropping increases with the queue length.

Summary



Summary

- The transport layer provides a logical end-to-end connection between processes
 - (De)multiplexing uses port numbers and IP addresses to associate packets with processes
 - Bit errors in received packets are detected using a checksum header field
- UDP is a “bare bones” transport protocol that does not provide any additional functionality
- TCP offers additional features, such as reliability, congestion control and flow control
 - Reliability is achieved by combining acknowledgments, timers, retransmissions and sequence numbers
 - TCP estimates the RTT to properly configure the retransmission timeout
 - Flow control allows the receiver to limit the transmission rate using the receive window header field
 - TCP adapts its congestion window when a timeout occurs or it receives three duplicate ACKs
- The contents of this part is covered in the **book** in Sections 3.1 – 3.7

End of Part 3